

Urban Resolve - Architecture & Flow Diagrams

This document contains the sequence and flow diagrams for the core systems of the Urban Resolve platform.

1. Authentication & Token Strategy (Hybrid Approach)

This sequence diagram illustrates the secure login process using short-lived Access Tokens in headers and long-lived Refresh Tokens in HttpOnly cookies.

```
sequenceDiagram
    actor User
    participant Frontend as React Frontend
    participant AuthCtrl as Auth Controller
    participant DB as MongoDB

    User->>Frontend: Enters Email & Password
    Frontend->>AuthCtrl: POST /login (credentials)
    AuthCtrl->>DB: findOne({ email })
    DB-->>AuthCtrl: Returns User Document
    AuthCtrl->>AuthCtrl: Validate Password (bcrypt.compare)
    AuthCtrl->>AuthCtrl: Generate Access Token (15m)
    AuthCtrl->>AuthCtrl: Generate Refresh Token (7d)
    AuthCtrl->>DB: Update User (Save Refresh Token)
    DB-->>AuthCtrl: Success
    AuthCtrl-->>Frontend: 200 OK + JSON (User Data, Access Token) + Cookie (Refresh Token)
    Note over Frontend, AuthCtrl: Subsequent API Requests
    Frontend->>AuthCtrl: Request with Authorization: Bearer <AccessToken>
```

2. Complaint Lifecycle (State Machine)

This state diagram shows the complete lifecycle of a complaint from the moment a citizen files it, through staff assignment, to final admin resolution.

```
stateDiagram-v2
    [*] --> PENDING : Citizen Submits Complaint
    PENDING --> IN_PROGRESS : Admin assigns to Staff
    IN_PROGRESS --> PENDING_REVIEW : Staff submits Resolution Request (with 2+ photos)
    PENDING_REVIEW --> RESOLVED : Admin Approves Proof
    PENDING_REVIEW --> IN_PROGRESS : Admin Rejects Proof (with reason)
    RESOLVED --> [*]
```

3. Secure Password Reset Flow (Zero-Trust)

This sequence diagram demonstrates the cryptographic process of resetting a password without storing raw tokens in the database.

```
sequenceDiagram
    actor User
    participant Frontend
    participant UserSvc as User Service
    participant DB as MongoDB
    participant Email as Email Provider (SendGrid/SMTP)

    User->>Frontend: Clicks "Forgot Password"
    Frontend->>UserSvc: POST /forgot-password (email)
    UserSvc->>UserSvc: Generate crypto Random Token (Cleartext)
    UserSvc->>UserSvc: Hash Token (SHA-256)
    UserSvc->>DB: Save Hashed Token & Expiry (15 mins)
    DB-->>UserSvc: Success
    UserSvc->>Email: Send Email with Cleartext Token URL
    Email-->>User: Delivers Email
    User->>Frontend: Clicks Email Link, Enters New Password
    Frontend->>UserSvc: POST /reset-password/:cleartextToken
    UserSvc->>UserSvc: Hash Cleartext Token (SHA-256)
    UserSvc->>DB: Find User by Hashed Token & Check Expiry
    DB-->>UserSvc: Returns User
    UserSvc->>UserSvc: Update Password & Wipe Tokens
    UserSvc->>DB: Save User
    DB-->>Frontend: 200 OK (Password Reset)
    Frontend-->>User: Redirects to Login Page
```

4. Distributed Cron Job & Notification Cleanup

This flow diagram illustrates how the background cleanup job utilizes a database lock (`System` collection) to prevent race conditions when running on multiple server instances simultaneously.

```
flowchart TD
    Start((Cron Triggers at 2:00 AM)) --> A[Fetch Last Cleanup Time from System DB]
    A --> B{Is Last Cleanup < 24 hrs ago?}
    B -- Yes (Another server ran it) --> C[Skip Execution & Exit]
    B -- No (First server to run) --> D[Update System DB Last Cleanup Time to NOW]
    D --> E[Calculate 7 Days Ago Timestamp]
    E --> F[MongoDB deleteMany: read_status=true AND updatedAt < 7 Days]
    F --> G[Log Deleted Count]
    G --> End((Process Complete))
```

5. Staff Upvote System (Atomic Operations)

This sequence diagram shows how the platform handles upvotes atomically to prevent "Lost Update" race conditions under heavy load.

```
sequenceDiagram
    actor User
    participant Controller as Complaint Controller
    participant DB as MongoDB

    User->>Controller: Click Upvote (POST /toggle-upvote)
    Controller->>DB: findOneAndUpdate({_id, "upvotes.users": {$ne: userId}}, {$addToSet})
    alt User is NOT in array
        DB-->>Controller: Returns updated document
        Controller-->>User: 200 OK
    else User is ALREADY in array
        DB-->>Controller: Returns null
        Controller->>DB: findOneAndUpdate({_id, {$pull: {"upvotes.users": userId}})
        DB-->>Controller: Returns updated document
        Controller-->>User: 200 OK (isUpvoted: false)
    end
```

